Urban Outfinders

Part 2: Indexing and Evaluation

Marc Mallol Èric Dürr Oriol Pont

Information Retrieval and Web Analytics, 2026

1 Introduction

This report covers Part 2 of our search engine: indexing the 28,080 products prepared in Part 1 and evaluating how well a TF-IDF ranking orders them. On top of the Part 1 records we complete the truncated brand names, build an inverted index, match queries conjunctively (a product must contain every query word), rank the matches by TF-IDF, and measure the ranking with seven metrics, on the two validation queries and on five test queries that we labelled ourselves.

Section 2 describes the index, the ranking and the test queries, Section 3 the metrics and their results, and Section 4 the main limitations of the current system and how we would address them. Section 5 concludes.

The code is at `https://github.com/uripont/urban-outfinders`, tag `IRWA-2026-part-2`. The `fashion_search` package holds the code (`index.py`, `ranking.py` and `evaluation.py`), and `notebooks/part_2.ipynb` runs it and produces every number in this report. The README explains how to reproduce them.

2 Indexing

The code is in the `fashion_search` package: `index.py` builds the inverted index and matches queries, and `ranking.py` ranks the matches. Section 2 of `notebooks/part_2.ipynb` runs them.

2.1 Inverted index

Brand names completed. Part 1 found that most brand names are cut off in the source data (*Pu*, *True Bl*, *REEB*), so a search for *puma* could not find a single Puma product. Product URLs start with the full brand (`/puma-solid-men-round-neck-...`), so before indexing we complete each brand whose letters run out inside a URL word with the rest of that word: *Pu* becomes *Puma* and *True Bl* becomes *True Blue*. This changes the brand of 26,798 of the 28,080 products. Brands that do not match the start of their URL are left as they are. We also strip the seller rating and notice glued to some seller names (*RetailNet4.5Seller changed. Check for...* becomes *RetailNet*).

Structure. The index maps each term to the products that contain it and how many times, $\{term : \{pid : count\}\}$. The counts are what TF-IDF needs; the set of pids is the posting list of the practical labs. Following Part 1, each product is indexed on one text that merges its title, description, product details and brand, all processed by the same `build_terms` as the queries. Category and sub-category are left out: they never contain query words and work better as filters. Table 1 gives the size of the result. Its vocabulary is very uneven: half the terms appear in at most 4 products, while the most common (*wear*, *wash*, *cotton*) appear in about 70% of them, mostly through the product details.

Table 1: Size of the inverted index.

Products	28,080
Terms	6,884
Postings (term, product pairs)	905,629
Median products per term	4
Terms in a single product	2,043

Conjunctive matching. A query matches the products that contain every one of its terms. We intersect the posting lists of the query terms, starting from the shortest, so each step can only shrink the set. Table 2 shows how each term narrows the validation queries: common terms such as *cotton* remove few products, while the garment words remove most.

Table 2: Products matching each term, and all terms so far, for the two validation queries (terms after pre-processing).

Query	Term	With the term	With all terms so far
1	women	13,807	13,807
	full	7,171	3,537
	sleev	18,031	3,479
	sweatshirt	1,419	671
	cotton	19,889	498
2	men	13,517	13,517
	slim	5,654	2,861
	jean	2,436	368
	blue	7,696	225

2.2 TF-IDF ranking

Each term t of a product or a query gets the weight

$$w_t = (1 + \log tf_t) \cdot \log \frac{N}{df_t},$$

where tf_t is how many times t appears in the product (or query), df_t is how many products contain t and $N = 28{,}080$. The logarithm on tf keeps a word repeated across the product details from dominating, and the IDF factor gives rare words more weight than words that most products share. A product's score is the cosine similarity between its weight vector and the query's, so longer product texts are not favoured just for having more words. Products with equal scores, often the same item listed by different sellers, are ordered by pid so results are reproducible.

Query 1 matches 498 products and query 2 matches 225. Table 3 shows their top five: the expected garments, from several brands, with scores that fall slowly because many products share almost the same text. Ranks 2 and 3 of query 2 tie and are ordered (by pid).

Runs to evaluate. `validation_labels.csv` judges only 20 products per query, and none of them is among the top 10 results of either query. Counting unjudged products as non-relevant would give every metric a score of zero and say nothing about the ranking. As in the practical labs, we evaluate how the engine ranks the judged products: each run is the list of judged products the engine returns, in its order. For query 1 the engine returns all 20 judged products, including the 13 relevant ones. For query 2 it returns only 8 of them, 6 of the 10 relevant ones, because the other four lack a query term (section 4).

Table 3: Top five TF-IDF results of the validation queries.

Rank	Product (pid)	Title	Brand	Score
Query 1: women full sleeve sweatshirt cotton				
1	SWSFZVTTQCB4SJ7F	Full Sleeve Solid Women Sweatshirt	Levis	0.730
2	SWSFZ2H4KMYXZXX7	Full Sleeve Solid Women Sweatshirt	ECKO Unltd	0.718
3	SWSFY38ADYPVZHYZ	Full Sleeve Printed Women Sweatshirt	ECKO Unltd	0.713
4	SWSFZVTNGM4HG8BC	Full Sleeve Printed Women Sweatshirt	Levis	0.711
5	SWSFYRKYAHH4HHSM	Full Sleeve Printed Women Sweatshirt	Steenbok	0.702
Query 2: men slim jeans blue				
1	JEAFSKYHZHSZZC9S	Slim Men Blue Jeans	ECKO Unltd	0.547
2	JEAFEKQZ4C2Z6GCX	Slim Men Blue Jeans	True Blue	0.536
3	JEAFEKQZQPYHHVGS	Slim Men Blue Jeans	True Blue	0.536
4	JEAFQF6JBUSEXHVF	Slim Men Blue Jeans	ECKO Unltd	0.520
5	JEAFSKYHSYMTKEJD	Slim Men Blue Jeans	ECKO Unltd	0.518

2.3 Test queries

The statement suggests choosing query words by their popularity. Words found in almost every product, such as *wear* or *cotton* (IDF below 0.4), barely tell products apart, while words found in a handful of products return almost nothing. We therefore built each query around a garment or brand word from the middle of the range, present in 300 to 6,000 products (IDF between about 1.5 and 4.5), and added the common words a user would type with it (gender, fit, fabric). Table 4 lists the five queries; each one tests something different.

Table 4: Test queries, with each term after pre-processing, its IDF and the number of products the query matches.

ID	Query	Terms (IDF)	Matches	What it tests
3	women printed kurta cotton	women (0.71), print (1.05), kurta (3.58), cotton (0.34)	40	One rare garment word among three common ones
4	men polo tshirt regular fit	men (0.73), polo (2.22), tshirt (0.77), regular (0.60), fit (0.95)	673	A long query with many matches and an explicit fit
5	slim fit jogger men	slim (1.60), fit (0.95), jogger (3.92), men (0.73)	33	A garment that products name in different ways
6	puma men tshirt	puma (3.56), men (0.73), tshirt (0.77)	165	Brand search, which needs the completed brands
7	women denim jacket	women (0.71), denim (2.76), jacket (3.23)	82	A material that is often only in the product details

Query 6 depends on the brand completion of Section 2.1. Without it, *puma* appears only where a title, description or detail mentions it (232 products), and the query matches 52 products. With it, all 798 Puma products carry the term, the query matches 165, and its top 20 results are all Puma men's t-shirts. Our first fifth query was *women ankle leggings*; we replaced it because the catalogue has no leggings (section 4).

3 Evaluation

3.1 Metrics

All metrics are implemented in `evaluation.py` and computed on the runs of Section 2.2. For a query, R is the number of judged products that are relevant and $rel_i \in \{0, 1\}$ is the label of the result at rank i. We report every metric at $K = 5$, 10 and 20: the first results a user sees, a typical first page, and all the products judged per query.

- **Precision@K** is the share of the top K results that are relevant: how much of what the user sees is useful. Positions the run does not fill count as misses, so a query whose run holds 8 products cannot score more than 8/20 at $K = 20$.
- **Recall@K** is the share of the R relevant products that appear in the top K: how much of what exists is found. R includes relevant products the engine never returns, so recall shows what AND matching loses. Precision and recall pull in opposite directions as K grows.
- **F1@K** is the harmonic mean of the two. It is high only when both are, which gives one number per query and K.
- **Average Precision@K** averages the precision at each rank, within the top K, that holds a relevant product: $AP@K = \frac{1}{\min(K,R)} \sum_{i \le K} P@i \cdot rel_i$. Unlike precision, it depends on the order inside the top K: a relevant product at rank 1 counts more than one at rank K. Dividing by $\min(K, R)$ lets a perfect top K score 1 even when more than K products are relevant.
- **Mean Average Precision** is the mean AP over the queries. For each query it uses the whole run and divides by R, so relevant products the engine never returns count against it. It summarises a system in one number.
- **Mean Reciprocal Rank** is the mean of $1/rank$ of the first relevant result. Only that result matters, which suits a user who wants one good product, and ignores everything after it.
- **NDCG@K** divides the discounted cumulative gain, $DCG@K = \sum_{i \le K} rel_i / \log_2(i+1)$, by the DCG of the best possible order of the judged products. Like AP it rewards relevant products near the top, but its logarithmic discount is smoother, so a late mistake costs less. With graded labels it would also reward the most relevant products first; ours are binary.

3.2 Validation queries

3.3 Test queries

Relevance labels. For each test query we judged its top 40 results, or all 33 matches for query 5, 193 products in all (`data/test_labels.csv`). We first judged only the top 20, but then every relevant product of a run was returned by construction and R@20 was always 1, so we extended the judgements to rank 40: relevant products ranked 21 to 40 now count in recall. Queries 3 and 5 are judged completely, since they match only 40 and 33 products. A rule on the words of each title proposed a first label; we then checked every product, reading its title, description and product details, and corrected 19 of the 193. Our criterion was that a product is relevant unless it contradicts something the query asks for. What the query leaves open is accepted: unstitched kurta fabric is relevant to *women printed kurta cotton*, and jogger-cut jeans to *slim fit jogger men*. What the query states must hold: two slim-fit polos are not relevant

Table 5: Evaluation of the validation queries.

Query	K	P@K	R@K	AP@K	F1@K	NDCG@K
1	5	1.000	0.385	1.000	0.556	1.000
	10	1.000	0.769	1.000	0.870	1.000
	20	0.650	1.000	0.976	0.788	0.994
2	5	0.800	0.400	0.800	0.533	0.869
	10	0.600	0.600	0.569	0.600	0.716
	20	0.300	0.600	0.569	0.400	0.716
MAP 0.772		MRR 1.000				

to *men polo tshirt regular fit*, and neither are jackets whose description only suggests wearing them with denims. Most corrections came from details the title leaves out: 15 jackets are denim only according to their product type, fabric or description. One case stayed borderline, a kurta whose pattern is listed as self design but whose description calls it printed; we kept it relevant. Table 6 gives the results.

Table 6: Evaluation of the test queries, with the relevant products among the judged ones (40 per query, 33 for query 5).

Query	Relevant	K	P@K	R@K	AP@K	F1@K	NDCG@K
3	40	5	1.000	0.125	1.000	0.222	1.000
		10	1.000	0.250	1.000	0.400	1.000
		20	1.000	0.500	1.000	0.667	1.000
4	38	5	0.800	0.105	0.710	0.186	0.830
		10	0.900	0.237	0.790	0.375	0.890
		20	0.900	0.474	0.787	0.621	0.889
5	10	5	1.000	0.500	1.000	0.667	1.000
		10	0.800	0.800	0.758	0.800	0.857
		20	0.500	1.000	0.923	0.667	0.978
6	40	5	1.000	0.125	1.000	0.222	1.000
		10	1.000	0.250	1.000	0.400	1.000
		20	1.000	0.500	1.000	0.667	1.000
7	34	5	1.000	0.147	1.000	0.256	1.000
		10	1.000	0.294	1.000	0.455	1.000
		20	1.000	0.588	1.000	0.741	1.000
MAP 0.964		MRR 1.000					

Analysis. Three queries have only relevant products in their top 20, so their precision, AP and NDCG are 1. In each, one rare word (*kurta*, *puma*, *denim* with *jacket*) defines the product, and TF-IDF gives it most of the weight. Query 7 also shows why the product details are indexed: the 15 jackets whose titles omit *denim* are still found, through their product type, fabric or description. Its six non-relevant results only appear from rank 31, among relevant ones. Recall is what tells these three queries apart from a perfect system: 40, 40 and 34 products are relevant, so a top 20 can hold at most 20 of them (R@20 of 0.500 to 0.588), and a user who stops at the first page sees only part of what exists.

The other two queries show how the metrics differ. In query 4, two slim-fit polos rank 3rd and 11th. Both contain *regular* somewhere in their text, so AND matching keeps them, and with an IDF of 0.60 the word barely changes the score, so missing the attribute the user asked for costs almost nothing. One mistake at rank 3 drops AP@5 to 0.710 while P@5 is still 0.800: AP punishes an early mistake more than precision does. Ranks 21 to 40 are all relevant. In query 5, precision falls from 1.000 to 0.500 between $K = 5$ and 20 while recall rises from 0.500 to 1.000, the usual trade-off. Its 10 relevant products all rank in the top 12, so AP@20 (0.923) and NDCG@20 (0.978) stay high although P@20 is low. The other ten results are t-shirts and boxers whose text mentions joggers. MRR is 1.000 for every validation and test query, since the first result is always relevant, so here it cannot tell the queries apart.

The test queries score higher than the validation ones (MAP 0.964 against 0.772), but not only because they are easier. The validation labels include relevant products the engine never returns, which lowers their recall and MAP. Ours cover the engine's own top 40, so a relevant product the engine misses or ranks below 40 is never counted. For queries 3 and 5 that loses nothing, since every match is judged; for queries 4, 6 and 7 recall is measured against the relevant products in the top 40 only and so overstates the true recall, which the 673, 165 and 82 matches suggest is lower.

4 Limitations and improvements

Section 4 of the notebook holds the evidence for each problem below.

AND matching is too strict. A product is lost if it lacks a single query term, however well it matches the rest. Four of the ten relevant products for *men slim jeans blue* are never returned: one is a *Tapered Fit Men Blue Jeans* without *slim*, and three are women's jeans that the labels consider relevant but that do not contain *men*. Common terms make this worse: *cotton* has an IDF of 0.35, so it barely changes the ranking, yet it still excludes every product that does not mention it. *Fix:* rank with a disjunctive (OR) query, so missing terms lower the score instead of removing the product, or drop query terms whose IDF is very low before matching.

Different words for the same thing. Products and users name the same garment differently, and a query finds only the products that use its word: *sweater* matches 149 products and *sweatshirt* 1,419, *sneakers* 780 and *shoes* 526, with little overlap. Compounds split the same way: *track pants* and *trackpants* are different terms. Part 1 unified a few such variants (*tee*, *t-shirt*); a fixed list does not scale. *Fix:* expand queries with synonyms from a hand-made garment list or from word embeddings (Part 3), and join compounds that appear both ways in the catalogue.

Queries for products the catalogue lacks. Our first fifth test query was *women ankle leggings*. No product title contains *legging*, and the Porter stemmer reduces *leggings* to *leg*, which matches track pants and jeans whose text mentions the leg. The engine returns 23 results, none of them relevant, and every metric is 0 whatever the order, so we replaced the query. *Fix:* tell the user when the most specific query word matches no title, and keep words such as *leggings* from being stemmed into a different word (an exception list, or a lemmatizer).

Attributes the user asks for weigh little. An explicit attribute such as *regular* in *men polo tshirt regular fit* is a common word, so it hardly affects the score, and AND matching accepts it anywhere in the text: two slim-fit polos rank among the top 11. *Fix:* recognise attribute words in the query and match them against the corresponding product detail (Fit, Fabric, Ideal For), as a filter or a boost.

One merged field. A term counts the same in the title as in a long list of product details, although the title says what the product is and the details mostly describe its fabric and care. *Fix:* index the fields separately and weight title matches more, as BM25F does.

Near-duplicates and no quality signal. Many products share a title and differ only in colour or seller, so the top 20 results of the validation queries contain only 3 and 4 distinct titles. TF-IDF also ignores rating, price and stock: four of the top 20 results of query 1 are out of stock. *Fix:* combine the text score with rating, discount and availability (the custom score of Part 3), and group or diversify near-identical products.

Incomplete relevance judgements. The validation labels judge 20 products per query, while the queries match 498 and 225 products, so the metrics only measure how the judged products are ordered, not the quality of what users would see first. Our test labels have the opposite problem: they cover only the engine's own top 40, so they cannot reveal relevant products it ranks lower or misses. *Fix:* judge the top results of every system being compared (pooling), so each one is measured on products the others found too.

5 Conclusions

Part 2 turns the Part 1 records into a working search engine. Completing the brand names from the product URLs fixes 26,798 brands and makes brand search possible: *puma men tshirt* goes from 52 to 165 matches. The inverted index holds 6,884 terms, conjunctive matching intersects their posting lists, and TF-IDF with cosine similarity orders the matches.

The ranking itself works well. The first result is relevant for every query (MRR 1.000), the validation queries reach a MAP of 0.772 and our test queries 0.964, with three of them free of mistakes in the top 20. The main weaknesses lie in matching and in what TF-IDF cannot see:

- **AND matching loses relevant products.** Four of the ten relevant products for *men slim jeans blue* lack one query word and are never returned, which caps its recall at 0.600.
- **Words must match exactly.** Synonyms (*sweater* and *sweatshirt*), compounds (*track pants* and *trackpants*) and the stemmer (*leggings* becomes *leg*) split or confuse the vocabulary.
- **Common words and attributes weigh little.** An attribute the user asks for, such as a regular fit, barely changes the score.
- **No signal beyond the text.** TF-IDF ignores fields, ratings, stock and near-duplicates.

These point to Part 3: a disjunctive or BM25 ranking that tolerates a missing word, field weights that favour the title, and a custom score that adds rating, discount and availability to text relevance. Finally, the validation labels cover few products and our test labels only the engine's top 40, so a fair comparison of the Part 3 rankings will need labels pooled from all of them.

Use of AI tools

We used Claude (Anthropic), through Claude Code, to review the notebook and this report. It helped write and debug the code, and helped draft this report from the notebook outputs. We checked every number against the notebook outputs and reviewed and edited the text ourselves.